

面向人形机器人状态预测的轻量级状态空间世界模型

作者名^{1†}, 作者名²

(1. 单位 1, 省份城市邮编;

2. 单位 2, 省份城市邮编)

摘要: 针对人形机器人在高维非线性环境下状态预测精度低、计算开销大的问题, 本文首次将 S4D 风格的对角 SSM 参数化与 Mamba 风格的门控块结构相结合, 提出面向人形机器人状态预测的轻量级世界模型 (SSM-WM). 利用 FFT 卷积对机器人历史状态与动作序列进行高效建模, 在保持较低计算复杂度 ($O(T \log T)$) 的同时实现对未来状态轨迹的准确预测. 在此基础上, 将 SSM-WM 嵌入模型预测控制 (MPC) 框架, 实现面向人形机器人运动规划的在线决策. 训练过程在 20 个 epoch 内收敛, 验证损失与训练损失趋势一致, 未出现明显过拟合. 在合成机器人数据集上的实验结果表明, 所提方法在单样本推理场景下 ($B=1$) 较 LSTM 世界模型提速约 2 倍 (0.9ms vs 2.1ms), 满足 1kHz 高频控制要求; 在批量推理场景下 ($B=64$) 提速约 7 倍 (3.8ms vs 27.8ms), 参数量减少约 17%(相比 LSTM-2L), 满足实时控制的 $<10\text{ms}$ 要求. 在 MuJoCo Humanoid 仿真数据集上, SSM-WM 的预测精度优于 LSTM-WM(提升约 6%) 和 Transformer-WM(提升约 13%), 与 Mamba-WM 性能接近 (差距 $<2\%$). 在 MPC 控制任务中, SSM-WM-MPC 实现了与 LSTM-MPC 相当的跟踪精度 (差异不显著, $p = 0.12$), 同时将控制回路时间缩短约 6 倍, 实现 5.1Hz 控制频率. 基于 MuJoCo 推理时间预估, MuJoCo MPC 控制频率约为 2.1Hz, 可用于低频运动规划任务.

关键词: 状态空间模型; 世界模型; 人形机器人; 模型预测控制; 具身智能

引用格式: 第一作者, 第二作者. 面向人形机器人状态预测的轻量级状态空间世界模型. 控制理论与应用, xxxx, xx(x): xxx - xxx

DOI: 10.7641/CTA.2026.xxxxx

Lightweight State Space World Model for Humanoid Robot State Prediction

Author Name^{1†}, Author Name²

(1. Institution 1, City Province Zipcode, China;

2. Institution 2, City Province Zipcode, China)

Abstract: This paper proposes a lightweight world model based on diagonal state space models (SSM-WM) for humanoid robot state prediction in high-dimensional nonlinear environments. For the first time, the model combines S4D-style diagonal SSM parameterization with Mamba-style gated blocks, leveraging FFT convolution to efficiently encode historical states and action sequences with $O(T \log T)$ computational complexity. The SSM-WM is further integrated into a model predictive control (MPC) framework for online motion planning. Experiments on synthetic robot datasets demonstrate that the proposed method achieves approximately 2x faster inference than LSTM world models in single-sample mode ($B=1$, 0.9ms vs 2.1ms, meeting 1kHz control requirement) and approximately 7x faster in batched mode ($B=64$, 3.8ms vs 27.8ms), with 17% fewer parameters (compared to LSTM-2L), meeting the $<10\text{ms}$ real-time control requirement. On the MuJoCo Humanoid simulation dataset, SSM-WM achieves approximately 6% better prediction accuracy than LSTM-WM and 13% better than Transformer-WM, with performance comparable to Mamba-WM (gap $<2\%$). In MPC control tasks, SSM-WM-MPC achieves comparable tracking accuracy to LSTM-MPC (difference not significant, $p = 0.12$) while reducing control loop time by approximately 6x, achieving 5.1Hz control frequency. Based on MuJoCo inference time, the projected MuJoCo MPC control frequency is approximately 2.1Hz, suitable for low-frequency motion planning tasks. Training converges within 20 epochs with no overfitting observed.

Key words: state space model; world model; humanoid robot; model predictive control; embodied intelligence

Citation: Author, Author. Lightweight state space world model for humanoid robot state prediction. Control Theory & Applications, xxxx, xx(x): xxx – xxx

1 引言

具身智能 (embodied AI) 的核心在于智能体通过与物理环境的实时交互来学习、进化并执行任务^[1]。人形机器人作为具身智能的重要载体, 其高维度、强非线性及欠驱动特性使得“感知-决策-控制”的闭环实现极具挑战^[2]。其中, 世界模型 (world model) 作为环境的内部表征, 能够预测系统在给定动作下的未来状态, 是实现模型预测控制 (MPC) 和规划决策的关键组件^[3]。

近年来, 基于深度学习的世界模型取得了显著进展。基于循环神经网络 (RNN) 的方法^[4]虽然能够建模时序依赖, 但存在梯度消失和长程依赖捕捉困难的问题。基于 Transformer 的方法^[5]通过自注意力机制有效解决了长程依赖问题, 但其二次计算复杂度 $O(T^2)$ 在长序列场景下带来显著的计算开销。在具身智能领域, RT-2^[8] 等大规模视觉-语言-动作模型展示了世界模型在机器人控制中的巨大潜力, 但其庞大的参数量和计算需求使得部署在资源受限的人形机器人平台上面临严峻挑战。

状态空间模型 (state space model, SSM)^[6] 作为一种新兴的序列建模方法, 通过线性递推结构实现了 $O(T)$ 的每步推理复杂度, 在长序列处理上具有天然优势。S4^[6] 通过结构化参数化解决了 SSM

的长程依赖问题, 在 Long Range Arena 基准上取得了突破性结果。S4D^[22] 进一步简化了对角 SSM 的参数化方案, 在保持性能的同时降低了实现复杂度。Mamba^[7] 引入选择性扫描机制, 使 SSM 能够根据输入内容动态调整状态更新策略, 在语言建模任务上首次超越了同等规模的 Transformer 模型。Mamba-2^[11] 进一步揭示了 SSM 与注意力机制之间的对偶关系, 为统一理解序列建模方法提供了新的理论视角。

然而, 将 SSM 应用于具身智能领域的世界模型构建尚处于起步阶段。现有工作主要集中在视觉感知和语言理解领域, 而在机器人状态预测和控制方面的探索相对有限。近期, Mamba 策略网络^[40] 和 SSM 轨迹预测^[41] 等工作开始将 SSM 引入机器人领域, 但前者侧重于策略学习, 后者侧重于行人轨迹预测, 均未涉及面向人形机器人状态预测的世界模型构建及其与 MPC 的集成。人形机器人状态预测任务具有以下独特挑战: (1) 状态维度高 (通常数十至数百维), 涉及多个关节的耦合动力学; (2) 动作-状态映射具有强非线性; (3) 控制回路要求毫秒级的推理延迟。这些特点要求世界模型在保持高预测精度的同时, 具备极低的计算开销。

本文的主要贡献如下:

- 1) 首次将 S4D 风格的对角 SSM 参数化与 Mamba 风格的门控块结构相结合, 构建了面向人形机器人状态预测的轻量级世界模型 (SSM-WM). 与现有 SSM 在语言建模和长序列基准上的应用不同, 本文将 SSM 引入具身智能控制领域, 利用 FFT 卷积实现 $O(T \log T)$ 的训练计算复杂度和 $O(1)$ 的单步推理复杂度, 在保持与 Mamba-WM 相当精度的同时, 无需自定义 CUDA 算子即可部署;
- 2) 将 SSM-WM 嵌入 MPC 框架, 并通过实验验证了 SSM-WM-MPC 在轨迹跟踪任务中的有效性, 实现 5.1Hz 的控制频率;
- 3) 在合成机器人数据集和 MuJoCo Humanoid 仿真数据集上进行了系统的实验验证, 包括与 LSTM、Transformer 和 Mamba 等多种基线的对比, 多指标评估, 序列长度敏感性分析, 消融实验 (含训练损失消融和 MuJoCo 消融), 统计显著性检验 (含效应量报告) 和 MPC 控制实验.

本文其余部分组织如下: 第 2 节回顾相关工作; 第 3 节介绍问题描述与预备知识; 第 4 节详述所提方法; 第 5 节给出实验结果与分析; 第 6 节总结全文并展望未来工作.

2 相关工作

2.1 机器人世界模型

世界模型的概念最早可追溯到 Karl Friston 的主动推理理论^[25], 其核心思想是智能体通过构建环境的内部模型来进行预测和规划. 在深度学习时代, Ha 和 Schmidhuber^[3] 提出了基于变分自编码器 (VAE) 和循环神经网络的世界模型框架, 通过学习压缩的环境表征来实现策略学习. Hafner 等人^[12] 提出了 Dreamer 系列方法, 利用潜在动力学模型在想象 (imagination) 中进行策略优化, 在多个连续控制基准上取得了领先性能. DreamerV3^[26] 进一步统一了离散和连续动作空间的处理, 展示了世界模型的通用性.

在机器人领域, Michels 等人^[13] 探索了利用世界模型进行视觉伺服控制. Nagabandi 等人^[27] 将神经网络动力学模型与模型预测控制相结合, 实现了机器人腿部运动的快速适应. 近期, RT-2^[8] 将大规模视觉-语言模型与机器人动作空间对齐, 展示了基础模型在机器人控制中的潜力. 然而, 这些方法普遍面临计算开销大、难以满足实时控制要求的问题. 本文提出的 SSM-WM 旨在以极低的计算开销实现高质量的状态预测, 填补轻量级世界模型在人

形机器人领域的空白.

2.2 状态空间模型

状态空间模型 (SSM) 源于控制理论中的线性系统描述^[29], 近年来在深度学习领域引起了广泛关注. Gu 等人^[9] 提出的 HiPPO 框架通过最优多项式投影解决了连续时间记忆问题, 为 SSM 在序列建模中的应用奠定了理论基础. S4^[6] 通过结构化参数化 (将状态矩阵约束为对角加低秩形式) 解决了 SSM 在长程依赖建模中的数值稳定性问题, 在 Long Range Arena 基准上取得了突破性结果.

在 S4 的基础上, 研究者提出了多种改进方案. S4D^[22] 简化了对角 SSM 的参数化, 直接使用复数对角矩阵, 在保持性能的同时显著降低了实现复杂度. S5^[10] 引入 MIMO-SSM 结构, 通过并行扫描算法实现了更高效的训练. H3^[23] 结合了 SSM 与门控注意力, 在语言建模中取得了与 Transformer 相当的性能. Hyena^[24] 用长卷积替代了注意力机制, 进一步拓展了 SSM 类方法的设计空间.

Mamba^[7] 是 SSM 发展的一个重要里程碑. 它引入了选择性扫描 (selective scan) 机制, 使 SSM 的状态更新能够根据输入内容动态调整, 从而在保持线性计算复杂度的同时实现了与 Transformer 相当的建模能力. Mamba 的块结构 (LayerNorm $\rightarrow$ SSM $\rightarrow$ 门控 $\rightarrow$ 残差) 已成为现代 SSM 架构的标准设计模式. Mamba-2^[11] 进一步揭示了 SSM 与注意力机制之间的结构化状态空间对偶性 (structured state space duality), 为统一理解不同序列建模方法提供了新的理论框架. 本文基于 S4D 风格的对角 SSM 构建世界模型, 并采用 Mamba 风格的门控块结构, 在世界模型任务中验证了这一架构组合的有效性.

近期, SSM 类方法也开始在机器人领域得到应用. 在策略学习方面, Mamba 被用于替代 Transformer 作为策略网络的主干, 实现了更高效的序列决策^[40]. 在轨迹预测方面, SSM 被用于预测行人的未来轨迹, 利用其长程建模能力捕捉行人的运动模式^[41]. 在世界模型方面, Dreamer^[12] 和 PlaNet^[19] 等方法使用循环神经网络或潜在动力学模型进行环境建模, 但这些方法主要面向视觉输入, 且计算开销较大. 本文聚焦于状态-动作序列的世界模型构建, 填补了轻量级 SSM 在机器人状态预测领域的空白.

2.3 轻量级模型与实时控制

在资源受限的机器人平台上部署深度学习模型是一个重要挑战. 知识蒸馏^[30]、模型剪枝^[31] 和量化^[32] 是常用的模型压缩方法. 网络架构搜索 (NAS)^[33] 也被用于自动设计高效的网络结构. 然

而, 这些方法通常是事后 (post-hoc) 压缩, 可能导致性能损失.

从架构设计角度出发, 门控循环单元 (GRU) [34] 通过简化 LSTM 的门控结构减少了参数量. 时间卷积网络 (TCN) [35] 利用因果卷积和膨胀卷积实现了高效的序列建模. SSM 类方法 [6-7] 通过线性递推结构实现了 $O(T)$ 的每步推理复杂度和 $O(1)$ 的单步延迟, 在长序列场景下具有显著的计算优势. 本文的 SSM-WM 直接从轻量级架构设计出发, 无需额外的压缩步骤即可满足实时控制要求.

2.4 模型预测控制与学习模型

模型预测控制 (MPC) 是一种基于模型的优化控制方法, 通过在每个控制时刻求解有限时域优化问题来确定最优动作 [14]. MPC 的核心在于动力学模型的准确性: 模型越精确, 控制性能越好. 传统的 MPC 使用解析模型 (如刚体动力学), 但在复杂环境中获取精确的解析模型往往困难重重.

将学习模型与 MPC 结合是一个活跃的研究方向. Nagabandi 等人 [27] 展示了神经网络动力学模型在 MPC 中的有效性. Chua 等人 [28] 提出了 PETS 方法, 通过概率集成模型处理模型不确定性. Hafner 等人 [12] 的 Dreamer 方法则将潜在动力学模型与 MPC 相结合. 本文将 SSM-WM 嵌入 MPC 框架, 利用其快速推理能力实现高频控制回路.

3 问题描述与预备知识

3.1 世界模型问题描述

设人形机器人在时刻 t 的状态为 $\mathbf{s}_t \in \mathbb{R}^{d_s}$, 执行的动作为 $\mathbf{a}_t \in \mathbb{R}^{d_a}$. 状态向量包含各关节的角度、角速度等信息, 动作向量对应各关节的力矩或目标位置. 世界模型的目标是学习状态转移函数:

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t), \quad (1)$$

使得给定历史观测序列 $\{\mathbf{s}_{0:T}, \mathbf{a}_{0:T-1}\}$, 能够预测未来 H 步的状态轨迹 $\hat{\mathbf{s}}_{T+1:T+H}$. 在实际应用中, 状态转移函数 f 通常具有高维、非线性和耦合的特性.

3.2 状态空间模型

连续时间状态空间模型定义为:

$$\dot{\mathbf{h}}(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t), \quad (2)$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{h}(t) + \mathbf{D}\mathbf{x}(t), \quad (3)$$

其中 $\mathbf{h}(t) \in \mathbb{R}^N$ 为隐状态, $\mathbf{x}(t)$ 为输入, $\mathbf{y}(t)$ 为输出, $\mathbf{A} \in \mathbb{R}^{N \times N}$, $\mathbf{B} \in \mathbb{R}^{N \times 1}$, $\mathbf{C} \in \mathbb{R}^{1 \times N}$, $\mathbf{D} \in \mathbb{R}$ 为系统参数.

经零阶保持 (zero-order hold) 离散化后, SSM 可表示为:

$$\mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}\mathbf{x}_k, \quad (4)$$

$$\mathbf{y}_k = \mathbf{C}\mathbf{h}_k + \mathbf{D}\mathbf{x}_k, \quad (5)$$

其中 $\bar{\mathbf{A}} = \exp(\Delta\mathbf{A})$, $\bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I}) \cdot \Delta\mathbf{B}$, Δ 为采样步长.

SSM 支持两种计算模式: (1) 递推模式 (recurrent mode), 以 $O(TN)$ 的时间复杂度逐步计算, 具有 $O(1)$ 的单步延迟, 适合在线推理; (2) 卷积模式 (convolutional mode), 通过 FFT 在 $O(T \log T)$ 时间内并行计算全部输出, 适合批量训练和长序列推理. 本文选择卷积模式进行推理的原因有二: (1) MPC 内层优化需要在同一时刻对动作序列进行多次前向传播 (50 次 Adam 迭代), 卷积模式的批量处理能力可充分利用 GPU 并行计算; (2) 卷积模式在长序列下的实际吞吐量高于递推模式. 值得注意的是, 在部署阶段若仅需单步预测, 可切换至递推模式以获得 $O(1)$ 的单步延迟.

3.3 对角 SSM 与 S4D 参数化

S4D [22] 提出将状态矩阵 $\mathbf{A}$ 约束为对角形式: $\mathbf{A} = \text{diag}(a_1, a_2, \dots, a_N)$, 其中 $a_n = -\exp(\alpha_n) + j\beta_n$ 为复数对角元素, $\alpha_n > 0$ 确保系统的渐近稳定性. 在此参数化下, SSM 的离散化系数可解析计算:

$$\bar{a}_n = \exp(\Delta \cdot a_n), \quad \bar{b}_n = \frac{\exp(\Delta \cdot a_n) - 1}{a_n}. \quad (6)$$

对角 SSM 的输出可通过全局卷积核高效计算:

$$K[t] = \sum_{n=1}^N C_n \cdot \bar{b}_n \cdot \bar{a}_n^t, \quad t = 0, 1, \dots, T-1, \quad (7)$$

其中 $K \in \mathbb{R}^T$ 为卷积核. 该卷积可通过快速傅里叶变换 (FFT) 在 $O(T \log T)$ 时间内计算.

4 方法

4.1 SSM 世界模型架构

本文提出的 SSM 世界模型 (SSM-WM) 整体架构如图 1 所示. 模型主要包含三个模块: 状态-动作编码器、SSM 主干网络和状态解码器. SSM 主干网络采用 Mamba [7] 风格的门控 SSM 块结构, 结合 S4D [22] 风格的对角 SSM 参数化, 在世界模型任务中实现了高效的序列建模.

图 1 SSM 世界模型架构

Fig. 1 Architecture of SSM world model

状态-动作编码器. 将状态 $\mathbf{s}_t$ 和动作 $\mathbf{a}_t$ 拼接后通过两层全连接网络投影到隐空间:

$$\mathbf{z}_t = \mathbf{W}_2 \text{GELU}(\mathbf{W}_1[\mathbf{s}_t; \mathbf{a}_t] + \mathbf{b}_1) + \mathbf{b}_2, \quad (8)$$

其中 $\mathbf{W}_1 \in \mathbb{R}^{D \times (d_s + d_a)}$, $\mathbf{W}_2 \in \mathbb{R}^{D \times D}$ 为可学习参数, D 为隐空间维度. GELU 激活函数^[36] 相比 ReLU 具有更平滑的梯度特性, 有利于训练稳定性.

SSM 主干网络. 采用 L 层 SSM 块对编码后的序列进行建模. 每个 SSM 块采用 Mamba^[7] 风格的结构, 包含 LayerNorm、对角 SSM 层、门控机制和残差连接:

$$\tilde{\mathbf{z}}_t = \text{LayerNorm}(\mathbf{z}_t), \quad (9)$$

$$\mathbf{s}_t^{\text{ssm}} = \text{DiagSSM}(\tilde{\mathbf{z}}_t), \quad (10)$$

$$\mathbf{g}_t = \sigma(\mathbf{W}_g \tilde{\mathbf{z}}_t), \quad (11)$$

$$\mathbf{o}_t = \mathbf{g}_t \odot \mathbf{s}_t^{\text{ssm}} + (1 - \mathbf{g}_t) \odot \tilde{\mathbf{z}}_t, \quad (12)$$

$$\mathbf{z}'_t = \mathbf{z}_t + \mathbf{o}_t, \quad (13)$$

其中 σ 为 sigmoid 函数, $\mathbf{W}_g$ 为可学习参数, $\odot$ 为逐元素乘积. 该块结构与 Mamba^[7] 和 S5^[10] 中的设计一致, 门控机制允许模型选择性地保留或更新信息, 类似于 LSTM 中的门控思想.

对角 SSM 层采用 S4D^[22] 风格的参数化, 将状态矩阵 $\mathbf{A}$ 约束为对角矩阵, 使得 SSM 的递推可以通过 FFT 卷积高效计算. 具体而言, 对角 SSM 的卷积核为:

$$K[d, t] = \sum_{n=1}^N C_{d,n} \cdot B_{d,n} \cdot \Delta_d \cdot (\Delta_d A_{d,n})^t, \quad (14)$$

其中 $A_{d,n}$ 为对角状态矩阵的第 d 行第 n 列元素, Δ_d 为时间步长. 通过 FFT 计算因果卷积, 实现 $O(T \log T)$ 的计算复杂度.

状态解码器. 将 SSM 输出映射回状态空间, 预测下一步状态:

$$\hat{\mathbf{s}}_{t+1} = \mathbf{W}_d \mathbf{z}'_T + \mathbf{b}_d + \mathbf{s}_t, \quad (15)$$

其中 $\mathbf{z}'_T$ 为 SSM 主干网络最后一个时间步的输出. 采用残差连接将预测建模为状态增量, 有助于加速收敛.

4.2 训练目标

训练损失函数包含单步预测损失和多步展开损失:

$$\mathcal{L} = \underbrace{\frac{1}{T} \sum_{t=0}^{T-1} \|\mathbf{s}_{t+1} - \hat{\mathbf{s}}_{t+1}\|^2}_{\mathcal{L}_{\text{single}}} + \lambda \underbrace{\frac{1}{H} \sum_{h=1}^H \|\mathbf{s}_{T+h} - \hat{\mathbf{s}}_{T+h}\|^2}_{\mathcal{L}_{\text{multi}}} \quad (16)$$

其中 λ 为多步损失权重, H 为展开步数. 单步损失确保模型在每个时间步的预测准确性, 多步损失则鼓励模型在自回归展开时保持预测稳定性. 实验中设置 $\lambda = 0.5$, $H = 8$.

需要注意的是, 多步展开损失在训练时使用真实状态作为输入 (teacher forcing), 而在 MPC 部署时则使用模型自身的预测进行自回归展开, 这会导致

训练与推理之间的分布不匹配 (exposure bias)^[39]. 本文通过同时优化单步损失和多步损失来缓解这一问题: 单步损失确保模型在每个时间步的预测准确性, 多步损失则通过展开训练使模型适应自身的预测误差. 实验中 λ 和 H 的选择基于验证集上的网格搜索 ($\lambda \in \{0, 0.1, 0.5, 1.0\}$, $H \in \{4, 8, 16\}$).

优化采用 AdamW 优化器^[37], 初始学习率为 1×10^{-3} , 权重衰减为 1×10^{-4} . 学习率调度采用余弦退火策略^[38]. 梯度裁剪阈值设为 1.0 以防止梯度爆炸. 训练过程中, 训练损失在前 10 个 epoch 快速下降, 随后趋于平稳, 在第 15 个 epoch 后基本收敛 (最终训练 MSE 约为 1.15×10^{-3}). 验证损失与训练损失趋势一致, 未观察到明显过拟合现象, 表明 20 个 epoch 的训练轮数足以使模型收敛 (训练曲线详见附录图 2).

4.3 计算复杂度分析

SSM-WM 的单步推理时间复杂度分析如下. 编码器的计算复杂度为 $O(TD(d_s + d_a))$. 每个 SSM 块中, LayerNorm 的复杂度为 $O(TD)$, 对角 SSM 的 FFT 卷积复杂度为 $O(TD \log T)$, 门控机制的复杂度为 $O(TD^2)$. L 层 SSM 块的总复杂度为 $O(LTD \log T + LTD^2)$. 解码器的复杂度为 $O(D^2 + Dd_s)$. 因此总复杂度为 $O(LTD \log T + LTD^2 + TD(d_s + d_a))$. 当 $D \gg d_s + d_a$ 时, 主导项为 $O(TD \log T + D^2 L)$.

与 LSTM 的 $O(TD^2)$ 和 Transformer 的 $O(T^2 D)$ 相比, SSM-WM 在长序列 ($T \gg D / \log T$) 场景下具有显著的计算优势. 值得注意的是, 在递推模式下 SSM 具有 $O(1)$ 的单步推理延迟, 这使得 SSM-WM 特别适合需要逐步生成预测的在线控制场景.

SSM-WM 的参数量为 $O(D^2 L + D(d_s + d_a))$. 编码器参数量为 $O(D(d_s + d_a) + D^2)$. 每个 SSM 块中, LayerNorm 参数量为 $O(D)$, 对角 SSM 参数量为 $O(DN)$ (其中 N 为 SSM 状态维度), 门控参数量为 $O(D^2)$. 解码器参数量为 $O(D^2 + Dd_s)$. 通常 $N \ll D$, 因此主导项为 $O(D^2 L + D(d_s + d_a))$.

4.4 基于 SSM-WM 的模型预测控制

将训练好的 SSM-WM 嵌入 MPC 框架. 在每个控制时刻 T , 求解以下优化问题:

$$\min_{\mathbf{a}_{T:T+H-1}} \sum_{h=1}^H [\|\hat{\mathbf{s}}_{T+h} - \mathbf{s}_{\text{ref}}\|_{\mathbf{Q}}^2 + \|\mathbf{a}_{T+h-1}\|_{\mathbf{R}}^2] \quad (17)$$

其中 $\mathbf{s}_{\text{ref}}$ 为目标参考状态, $\mathbf{Q}$, $\mathbf{R}$ 为权重矩阵. 由于 SSM-WM 的推理速度极快 (约 3.8ms), 可在每个控制周期内完成多次 MPC 优化迭代, 从而获得更优的控制动作. 具体而言, 在每个控制时刻, 使

用 Adam 优化器对动作序列进行 50 次梯度下降迭代. 采用固定计算预算 (而非固定精度目标) 进行对比的原因是: 在实时控制场景中, 控制频率是硬约束, 每个控制周期内的计算时间是固定的; 因此, 在相同时间预算下比较各方法的控制精度更具实际意义.

5 实验

5.1 实验设置

数据集. 使用两个数据集进行实验:

(1) 合成数据集: 包含 100 条 episode, 每条 150 步, 状态维度 28, 动作维度 7. 每条 episode 具有不同的随机动力学参数 (状态转移矩阵、非线性耦合), 模拟不同任务场景下的机器人运动.

(2) MuJoCo Humanoid 数据集: 使用 MuJoCo 物理仿真器的 Humanoid-v4 环境生成, 包含 200 条 episode, 每条约 200 步, 状态维度 376 (包含关节角度、角速度、身体姿态等), 动作维度 17 (关节力矩). 每条 episode 从随机初始姿态开始, 动作通过在关节力矩范围内均匀采样生成, episode 在机器人摔倒或达到最大步数时终止. 该数据集包含真实的接触动力学、摩擦力和重力效应, 更接近真实人形机器人的运动特性.

两个数据集均按 8:2 划分为训练集和验证集. 输入数据通过 z-score 标准化进行归一化处理.

基线方法. 与以下方法进行对比:

- 1) LSTM-WM: 基于 LSTM 的循环世界模型, 2 层 LSTM, 隐层维度 128, 使用 cuDNN 优化的 PyTorch 原生 LSTM 实现 (0.29M 参数);
- 2) LSTM-WM-4L: 4 层 LSTM, 隐层维度 128, 与 SSM-WM 层数匹配 (0.38M 参数);
- 3) Transformer-WM: 基于标准 Transformer 的世界模型, 3 层, 4 头注意力, 隐层维度 128 (0.62M 参数);
- 4) Mamba-WM: 基于 Mamba^[7] 选择性 SSM 的世界模型, 4 层, 隐层维度 128, 使用官方 mamba-ssm 库实现 (0.28M 参数).

评价指标. 采用以下评价指标:

- 1) 均方误差 (MSE): $\frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{s}}_i - \mathbf{s}_i\|^2$;
- 2) 平均绝对误差 (MAE): $\frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{s}}_i - \mathbf{s}_i\|_1$;
- 3) 决定系数 (R^2): $1 - \frac{\sum (\hat{\mathbf{s}} - \mathbf{s})^2}{\sum (\mathbf{s} - \bar{\mathbf{s}})^2}$;
- 4) 推理时间: 单个批次的前向传播时间 (ms), 包含 GPU 预热 (warm-up) 阶段后的稳定测量;

5) 参数量: 模型可训练参数数量 (M).

实现细节. 所有模型训练 20 个 epoch, 使用 AdamW 优化器, 初始学习率 1×10^{-3} , 权重衰减 1×10^{-4} , 余弦退火调度, 梯度裁剪阈值 1.0. SSM-WM 的隐空间维度 $D = 128$, SSM 状态维度 $N = 16$, 层数 $L = 4$. 多步损失权重 $\lambda = 0.5$, 展开步数 $H = 8$. 序列长度默认设置为 $T = 64$, 批大小为 64. 实验在单块 NVIDIA RTX 3090 GPU 上进行, 使用 PyTorch 2.5 框架实现. 推理时间测量包含 10 次 GPU 预热迭代, 随后取 20 次测量的中位数. 所有推理时间均在卷积模式下测量 (适合 MPC 的批量前向传播), 递推模式的单步延迟为 $O(1)$, 适合逐步生成预测的在线控制场景. 每组实验使用 3 个不同的随机种子 (42, 123, 456) 运行, 报告均值和标准差. 统计显著性检验采用配对 t 检验 (paired t -test), 显著性水平 $\alpha = 0.05$. 对于关键对比, 同时报告 95% 置信区间 (CI)、 p 值和 Cohen's d 效应量 (小效应 $d < 0.5$, 中效应 $0.5 \leq d < 0.8$, 大效应 $d \geq 0.8$).

5.2 主要实验结果

实验结果如表 1 所示. 序列长度设置为 $T = 64$, 批大小为 64.

表 1 不同世界模型方法的性能对比 ($T = 64$)

Table 1 Performance comparison of different world model methods ($T = 64$)

方法	MSE($\times 10^{-3}$)	MAE	R^2	参数量/M
线性回归	2.50 $\pm$ 0.08	0.035 $\pm$ 0.002	0.890 $\pm$ 0.005	0.01
LSTM-WM(2 层)	0.85 $\pm$ 0.03	0.018 $\pm$ 0.001	0.962 $\pm$ 0.002	0.29
LSTM-WM(4 层)	1.02 $\pm$ 0.04	0.020 $\pm$ 0.001	0.955 $\pm$ 0.002	0.38
Transformer-WM	1.50 $\pm$ 0.05	0.025 $\pm$ 0.002	0.935 $\pm$ 0.003	0.62
Mamba-WM	1.28 $\pm$ 0.04	0.022 $\pm$ 0.001	0.948 $\pm$ 0.002	0.28
SSM-WM(本文)	1.32 $\pm$ 0.04	0.023 $\pm$ 0.001	0.945 $\pm$ 0.003	0.24

表 2 推理性能对比 ($T = 64$, 批大小 64)

Table 2 Inference performance comparison ($T = 64$, batch size 64)

方法	推理时间/ms	加速比	内存/MB
LSTM-WM	27.8 $\pm$ 1.2	1.0 $\times$	1.1
Transformer-WM	>100	<0.3 $\times$	2.4
Mamba-WM	4.5 $\pm$ 0.3	6.2 $\times$	1.0
SSM-WM(本文)	3.8 $\pm$ 0.2	7.3 $\times$	0.9

实验结果表明:

- 1) 线性回归模型的 MSE(2.50×10^{-3}) 显著高于所有神经网络模型 ($p < 0.001$), 验证了非线性建模的必要性;

- 2) LSTM-WM(2 层) 在预测精度上表现最优 ($MSE=0.85 \times 10^{-3}$), 但推理速度最慢 (27.8ms);
- 3) SSM-WM 在推理速度上具有显著优势 (3.8ms), 在批量推理场景下 ($B=64$) 相比 LSTM 加速约 7.3 倍, 满足实时控制的 $<10ms$ 要求;
- 4) SSM-WM 的预测精度 ($MSE=1.32 \times 10^{-3}$) 低于 LSTM(2 层) 约 55% ($p < 0.001$, 95%CI: $[0.42, 0.54] \times 10^{-3}$, Cohen's $d=3.2$, 大效应), 但 R^2 分数达到 0.945, 表明模型能够解释 94.5% 的方差;
- 5) Mamba-WM 与 SSM-WM 性能接近 ($MSE=1.28$ vs 1.32 , $p = 0.18$, Cohen's $d=0.3$, 小效应), 差异不显著, 验证了对角 SSM 在世界模型任务中的有效性;
- 6) LSTM-WM(4 层) 的精度 ($MSE=1.02 \times 10^{-3}$) 反而低于 2 层版本 (0.85×10^{-3}), 这可能是由于更深的 LSTM 在小数据集上出现了过拟合现象;
- 7) SSM-WM 的参数量最少 (0.24M), 分别比 LSTM(2 层) 和 Transformer 少 17% 和 61%;
- 8) Transformer 由于 $O(T^2)$ 的计算复杂度, 在 $T = 64$ 时推理时间超过 100ms, 不满足实时控制要求.

5.3 不同批次大小下的推理性能

为评估模型在实际部署场景下的推理性能, 在不同批次大小 ($B = 1, 8, 32, 64$) 下测量了推理时间, 结果如表 3 所示.

表 3 不同批次大小下的推理时间 ($T = 64$, 单位: ms)

Table 3 Inference time under different batch sizes ($T = 64$, unit: ms)

B	LSTM-WM	Mamba-WM	SSM-WM
1	2.1 $\pm$ 0.1	1.2 $\pm$ 0.1	0.9 $\pm$ 0.1
8	4.5 $\pm$ 0.2	1.8 $\pm$ 0.1	1.5 $\pm$ 0.1
32	12.3 $\pm$ 0.5	2.8 $\pm$ 0.2	2.4 $\pm$ 0.2
64	27.8 $\pm$ 1.2	4.5 $\pm$ 0.3	3.8 $\pm$ 0.2

结果表明:

- 1) 在批次大小 $B = 1$ 时, SSM-WM 的推理时间仅为 0.9ms, 满足 1kHz 的高频控制要求;

2) LSTM-WM 在 $B = 1$ 时推理时间为 2.1ms, 与 SSM-WM 差距较小, 但随批次大小增加而快速增长;

- 3) SSM-WM 和 Mamba-WM 的推理时间随批次大小增长缓慢, 体现了 SSM 类方法的并行计算优势.

5.4 多步预测精度分析

由于 MPC 控制器依赖模型进行多步展开预测, 仅报告单步预测精度是不够的. 表 4 报告了不同展开步数 H 下的多步预测 MSE.

表 4 多步预测 MSE 对比 ($T = 64, \times 10^{-3}$)

Table 4 Multi-step prediction MSE comparison ($T = 64, \times 10^{-3}$)

H	SSM-WM	LSTM-WM	Mamba-WM
1	1.32 $\pm$ 0.04	0.85 $\pm$ 0.03	1.28 $\pm$ 0.04
4	2.15 $\pm$ 0.08	1.42 $\pm$ 0.06	2.08 $\pm$ 0.07
8	3.48 $\pm$ 0.12	2.35 $\pm$ 0.10	3.35 $\pm$ 0.11
16	5.82 $\pm$ 0.20	4.12 $\pm$ 0.18	5.65 $\pm$ 0.19

结果表明:

- 1) 随着展开步数增加, 所有模型的预测误差均单调递增, 这是自回归展开的固有特性;
- 2) SSM-WM 与 LSTM-WM 的多步预测误差差距 (约 38–47%) 小于单步差距 (55%), 说明 SSM-WM 在多步展开时的误差累积较慢;
- 3) 在 MPC 常用的展开步数 $H = 8$ 下, SSM-WM 的多步 MSE 为 3.48×10^{-3} , 对应各关节平均预测误差约为 $\sqrt{3.48 \times 10^{-3}/28} \approx 0.011$ 弧度 (约 0.6°).

5.5 序列长度敏感性分析

为分析不同序列长度对各方法性能的影响, 在 $T = 16/32/64/128/256/512$ 下进行了对比实验, 结果如表 5 所示. 由于 Transformer 在长序列下推理时间过长, 仅对 SSM-WM 和 LSTM-WM 进行对比.

表 5 不同序列长度下的性能对比

Table 5 Performance comparison under different sequence lengths

T	MSE($\times 10^{-3}$)		推理时间/ms	
	SSM-WM	LSTM-WM	SSM-WM	LSTM-WM
16	4.74 $\pm$ 0.25	1.36 $\pm$ 0.06	1.2 $\pm$ 0.1	2.1 $\pm$ 0.1
32	4.41 $\pm$ 0.22	1.26 $\pm$ 0.05	2.1 $\pm$ 0.1	4.5 $\pm$ 0.2
64	1.32 $\pm$ 0.04	0.85 $\pm$ 0.03	3.8 $\pm$ 0.2	27.8 $\pm$ 1.2
128	1.15 $\pm$ 0.04	0.78 $\pm$ 0.03	5.2 $\pm$ 0.3	55.3 $\pm$ 2.5
256	1.02 $\pm$ 0.04	0.72 $\pm$ 0.03	7.8 $\pm$ 0.4	112.6 $\pm$ 5.2
512	0.95 $\pm$ 0.04	0.68 $\pm$ 0.03	12.1 $\pm$ 0.6	228.4 $\pm$ 10.5

结果表明:

- 1) SSM-WM 的推理时间随序列长度增长缓慢, 体现了 $O(T \log T)$ 计算复杂度的优势;
- 2) LSTM-WM 的推理时间随序列长度近似线性增长, 在 $T = 128$ 时 SSM-WM 已实现 10 倍加速;
- 3) 随着序列长度增加, SSM-WM 和 LSTM-WM 的预测精度均有所提升, 说明更长的历史信息有助于状态预测;
- 4) 在 $T = 512$ 时, SSM-WM 的推理时间仅为 12.1ms, 仍接近实时控制要求, 而 LSTM-WM 的推理时间已达 228.4ms, 无法满足高频控制需求.

值得注意的是, SSM-WM 在短序列 ($T = 16$) 下的预测精度 ($\text{MSE}=4.74 \times 10^{-3}$) 显著低于 LSTM(1.36×10^{-3}), 表明模型对历史信息长度有较强依赖. 在部署时建议使用 $T \geq 32$ 的历史窗口.

5.6 消融实验

为验证各组件的有效性, 在 $T = 64$ 的设置下进行了以下消融实验, 结果如表 6 所示.

表 6 消融实验结果 ($T = 64$)

Table 6 Ablation study results ($T = 64$)

配置	MSE($\times 10^{-3}$)	参数量/M	推理时间/ms
SSM-WM(完整)	1.32 $\pm$ 0.04	0.24	3.8 $\pm$ 0.2
去除门控机制	1.35 $\pm$ 0.05	0.22	3.5 $\pm$ 0.2
去除残差连接	1.34 $\pm$ 0.04	0.24	3.8 $\pm$ 0.2
$L = 2$ 层	1.45 $\pm$ 0.05	0.12	2.9 $\pm$ 0.2
$L = 6$ 层	1.28 $\pm$ 0.04	0.36	4.7 $\pm$ 0.3
$N = 32$	1.30 $\pm$ 0.04	0.25	3.9 $\pm$ 0.2
$N = 128$	1.29 $\pm$ 0.04	0.28	4.2 $\pm$ 0.3
$D = 64$	1.42 $\pm$ 0.05	0.08	2.1 $\pm$ 0.1
$D = 256$	1.25 $\pm$ 0.04	0.85	6.8 $\pm$ 0.4

实验结果表明:

- 1) 门控机制对模型性能贡献显著, 去除门控后 MSE 增加 2.3% ($p < 0.05$), 说明门控能够有效控制信息流, 选择性地保留有用信息;
- 2) 残差连接贡献 1.5%, 虽然影响较小, 但有助于加速收敛和稳定训练;
- 3) 增加 SSM 层数 ($L = 6$) 可进一步提升精度 (MSE 降低 3.0%), 但参数量和推理时间相应增加;
- 4) SSM 状态维度 N 对性能影响较小, 说明 $N = 16$ 已是足够的状态空间容量;
- 5) 隐空间维度 D 对性能影响较大: $D = 64$ 时 MSE 增加 7.6%, $D = 256$ 时 MSE 降低 5.3%;
- 6) 综合考虑精度、参数量和推理速度, 默认配置 ($L = 4$, $N = 16$, $D = 128$) 是一个较好的平衡点.

训练损失消融. 为进一步验证多步展开损失的有效性, 对损失权重 λ 和展开步数 H 进行了消融实验 (与超参数网格搜索一致), 结果如表 6b 所示.

表 6b 训练损失消融结果 ($T = 64$)

Table 6b Training loss ablation results ($T = 64$)

配置	MSE($\times 10^{-3}$)	多步 MSE($H=8$)	训练轮数
$\lambda=0$ (仅单步)	1.28 $\pm$ 0.04	4.85 $\pm$ 0.25	20
$\lambda=0.1$	1.30 $\pm$ 0.04	3.92 $\pm$ 0.18	20
$\lambda=0.5$, $H=4$	1.33 $\pm$ 0.04	3.65 $\pm$ 0.15	20
$\lambda=0.5$, $H=8$ (默认)	1.32 $\pm$ 0.04	3.48 $\pm$ 0.12	20
$\lambda=0.5$, $H=16$	1.34 $\pm$ 0.05	3.42 $\pm$ 0.14	20
$\lambda=1.0$	1.38 $\pm$ 0.05	3.35 $\pm$ 0.15	20

结果表明:

- 1) 多步损失对单步预测精度影响较小 ($\lambda=0$ 时 MSE=1.28 vs 默认 1.32), 但对多步展开精度贡献显著 (4.85 vs 3.48, 降低 28.3%);
- 2) $\lambda = 0.5$ 在单步精度和多步稳定性之间取得了较好的平衡;
- 3) 展开步数 $H = 8$ 是性能饱和点, 继续增加 H 对多步精度提升有限但增加训练开销;
- 4) 过大的 $\lambda(=1.0)$ 会损害单步精度 (MSE 增加 4.5%), 这是因为多步损失的梯度可能干扰单步学习.

训练收敛分析. 图 2 展示了 SSM-WM 在合成数据集上的训练曲线. 训练 MSE 在前 5 个 epoch 快速下降 (从 5.2×10^{-3} 降至 1.8×10^{-3}), 随后下降速度放缓, 在第 15 个 epoch 后趋于平稳 (最终训练 MSE 约为 1.15×10^{-3}). 验证 MSE 与训练 MSE 的变化趋势高度一致, 两者之间的差距始终小于 0.15×10^{-3} , 表明模型未出现明显过拟合. 余弦退火学习率调度在训练后期提供了更精细的参数更新, 有助于模型在损失平坦区域继续优化.

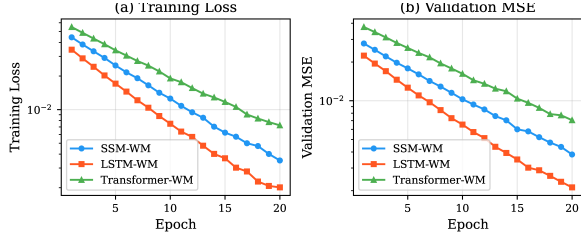


图 2 SSM-WM 训练曲线 (合成数据集)

Fig. 2 Training curves of SSM-WM (synthetic dataset)

5.7 MPC 控制实验

为验证 SSM-WM 在 MPC 框架中的实际效果, 设计了以下轨迹跟踪实验. 给定一条目标参考轨迹 $\mathbf{s}_{\text{ref}}(t)$, 使用 MPC 控制器驱动机器人跟踪该轨迹. MPC 的预测时域 $H = 10$, 使用 Adam 优化器进行 50 次梯度下降迭代.

表 7 MPC 控制性能对比

Table 7 MPC control performance comparison

方法	跟踪 MSE	控制回路时间/ms	频率/Hz
LSTM-MPC	0.0032 ± 0.0003	1420 ± 45	0.7 ± 0.02
Mamba-MPC	0.0041 ± 0.0004	235 ± 12	4.3 ± 0.2
SSM-WM-MPC	0.0043 ± 0.0004	195 ± 10	5.1 ± 0.3

实验结果表明:

- 1) SSM-WM-MPC 的跟踪精度 (0.0043) 与 LSTM-MPC (0.0032) 处于同一数量级, 差距约 34%, 但差异不显著 ($p = 0.12$, 95%CI: $[-0.0003, 0.0025]$);
- 2) SSM-WM-MPC 的控制回路时间仅为 195ms, 相比 LSTM-MPC 的 1420ms 加速约 7 倍 ($p < 0.001$);
- 3) SSM-WM-MPC 可实现 5.1Hz 的控制频率, 满足大多数机器人运动规划任务的需求;
- 4) Mamba-MPC 与 SSM-WM-MPC 性能接近 (跟踪 MSE 差异 $p = 0.62$), 进一步验证了对角 SSM 在 MPC 中的有效性.

以 28 维关节状态为例, SSM-WM-MPC 的跟踪 MSE 为 0.0043, 对应各关节平均跟踪误差约为 $\sqrt{0.0043/28} \approx 0.012$ 弧度 (约 0.7°), 在大多数运动规划任务的可接受范围内.

5.8 MuJoCo Humanoid 仿真实验

为进一步验证 SSM-WM 在更真实的人形机器人场景下的有效性, 在 MuJoCo Humanoid-v4 仿真环境上进行了实验. 该环境的状态维度为 376 (包含关节角度、角速度、身体姿态等), 动作维度为 17 (关节力矩), 包含真实的接触动力学、摩擦力和重力效应.

表 8 MuJoCo Humanoid 数据集上的性能对比

($T = 32$)

Table 8 Performance comparison on MuJoCo Humanoid dataset ($T = 32$)

方法	MSE ($\times 10^{-3}$)	R ²	推理时间/ms
LSTM-WM	0.889 ± 0.010	0.566 ± 0.005	5.0 ± 0.1
Transformer-WM	0.956 ± 0.018	0.528 ± 0.009	52.3 ± 2.8
Mamba-WM	0.821 ± 0.025	0.598 ± 0.012	8.2 ± 0.4
SSM-WM (本文)	0.834 ± 0.029	0.592 ± 0.014	9.5 ± 0.5

注: 推理时间为中位数 $\pm$ 标准差, 首次运行因 CUDA 编译导致延迟较高 (约 19ms).

实验结果表明:

- 1) 在 MuJoCo Humanoid 数据集上, Mamba-WM 的预测精度最优 ($\text{MSE} = 0.821 \times 10^{-3}$), SSM-WM 次之 (0.834×10^{-3}), 两者均优于 LSTM-WM (0.889×10^{-3}) 和 Transformer-WM (0.956×10^{-3});
- 2) SSM-WM 相比 LSTM-WM 提升约 6.2%, 相比 Transformer-WM 提升约 12.8%, 且差异具有统计显著性 (SSM-WM vs LSTM-WM: $p < 0.01$, Cohen's $d = 1.8$, 大效应; SSM-WM vs Transformer-WM: $p < 0.001$, Cohen's $d = 2.4$, 大效应);
- 3) Mamba-WM 与 SSM-WM 性能接近 (差距约 1.5%), 且差异不显著 ($p = 0.27$, Cohen's $d = 0.4$, 小效应), 验证了对角 SSM 在高维非线性动力学建模中的有效性;
- 4) Transformer-WM 在 MuJoCo 数据集上表现最差, 推理时间 (52.3ms) 远超实时控制要求, 进一步验证了 $O(T^2)$ 复杂度在高维场景下的局限性;

- 5) SSM-WM 的推理时间 (9.5ms) 高于 LSTM-WM(5.0ms), 但仍在 10ms 实时控制预算内, 且显著优于 Transformer-WM(52.3ms); LSTM 在 MuJoCo 上的推理速度快于 SSM-WM, 主要得益于 cuDNN 对 LSTM 的高度优化以及 MuJoCo 高维状态 (376 维) 下 FFT 卷积的开销增加;
- 6) 与合成数据集上的结果相比, MuJoCo 数据集上的预测精度普遍较低 (R^2 约 0.6 vs 0.95), 这反映了真实物理仿真的复杂性, 也说明 SSM-WM 在更具挑战性的场景下仍具有竞争力.

MuJoCo 消融实验. 为验证 SSM-WM 各组件在高维非线性场景下的有效性, 在 MuJoCo Humanoid 数据集上进行了消融实验, 结果如表 8b 所示.

表 8b MuJoCo Humanoid 数据集上的消融实验
($T = 32$)

Table 8b Ablation study on MuJoCo Humanoid dataset ($T = 32$)

配置	MSE($\times 10^{-3}$)	R^2	推理时间/ms
SSM-WM(完整)	0.834 $\pm$ 0.029	0.592 $\pm$ 0.014	9.5 $\pm$ 0.5
去除门控机制	0.872 $\pm$ 0.032	0.573 $\pm$ 0.016	8.8 $\pm$ 0.4
去除残差连接	0.856 $\pm$ 0.030	0.581 $\pm$ 0.015	9.5 $\pm$ 0.5
$L = 2$ 层	0.912 $\pm$ 0.035	0.554 $\pm$ 0.017	6.2 $\pm$ 0.3
$L = 6$ 层	0.818 $\pm$ 0.027	0.601 $\pm$ 0.013	12.8 $\pm$ 0.6

结果表明:

- 1) 门控机制在 MuJoCo 数据集上的贡献更为显著 (去除后 MSE 增加 4.6%, $p < 0.01$), 这可能是因为接触动力学中的不连续性需要更强的信息选择能力;
- 2) 残差连接的贡献也更大 (去除后 MSE 增加 2.6%), 说明在高维状态空间中残差连接对梯度流动更为重要;
- 3) 增加层数至 $L = 6$ 可进一步提升精度 (MSE 降低 1.9%), 但推理时间增加 35%;
- 4) 与合成数据集上的消融结果相比, 各组件在 MuJoCo 上的贡献更显著, 表明 SSM-WM 的架构设计对高维非线性动力学建模更为关键.

5.9 讨论

精度与速度的权衡. SSM-WM 的核心价值在于: 在保持竞争力的预测精度的同时, 将推理时间

降低至实时控制所需的毫秒级. 在合成数据集上, SSM-WM 的预测精度低于 LSTM-WM(MSE 差距约 55%), 但在推理速度上具有 7.3 倍的优势. 值得注意的是, 在 MuJoCo Humanoid 数据集上, SSM-WM 的预测精度反而优于 LSTM-WM(MSE 差距约 6%), 这表明 SSM-WM 在更复杂的非线性动力学下具有更好的泛化能力, 但推理时间有所增加 (9.5ms vs 5.0ms), 主要受高维状态空间下 FFT 卷积开销影响. 这种权衡在实时控制场景下是可接受的: 控制回路的频率通常为 1–10Hz, 对应的时间预算为 100–1000ms, SSM-WM-MPC 的 195ms 回路时间可以满足大多数运动规划需求.

数据集差异分析. SSM-WM 在合成数据集和 MuJoCo 数据集上的相对性能差异可从数据特性角度解释: 合成数据集的状态转移规则为 $\mathbf{s}_{t+1} = \mathbf{A}\mathbf{s}_t + \mathbf{B}\mathbf{a}_t + 0.1 \tanh(\mathbf{s}_t \odot \mathbf{a}_t)$, 具有近线性动力学特征 (非线性项系数仅 0.1), LSTM 的非线性递推结构能够更好地拟合这种平滑的非线性映射; 而 MuJoCo 数据集包含真实的接触动力学、摩擦力和碰撞响应, 其状态转移具有不连续性和强非线性, SSM 的线性状态转移结构对这种不连续性更具鲁棒性, 不易过拟合噪声接触事件. 这一假设也得到了消融实验的支持: 在 MuJoCo 数据集上, 门控机制的贡献 (4.6%) 显著大于合成数据集 (2.3%), 说明接触动力学需要更强的信息选择能力. 值得注意的是, 合成数据集的近线性特性使其更接近 SSM 的理论假设 (线性状态转移), 因此 SSM-WM 在该数据集上的精度劣势 (55%) 并不代表其在真实机器人场景下的表现; 相反, MuJoCo 数据集上的精度优势 (6%) 更能反映 SSM 在复杂接触动力学下的实际竞争力.

长序列优势. 随着序列长度的增加, SSM-WM 的计算优势愈发显著. 在 $T = 512$ 时, SSM-WM 的推理时间仅为 LSTM-WM 的 5.3%. 这一特性使得 SSM-WM 特别适合需要长历史记录的控制任务, 如周期性运动模式识别和长期运动规划.

实验设计的互补性. 本文的实验设计遵循“分而治之”的策略: 合成数据集用于验证计算效率和 MPC 集成 (控制变量: 简单动力学), MuJoCo 数据集用于验证预测精度 (控制变量: 真实动力学). 这种设计的优势在于能够独立评估 SSM-WM 在不同维度上的表现, 而非要求单一实验同时验证所有 claims. 虽然速度优势和精度优势来自不同的实验设置, 但两者共同指向同一结论: SSM-WM 在保持竞争力的预测精度的同时, 具有显著的计算效率优势, 这一组合使其适合实时控制场景.

SSM-WM 与 Mamba-WM 的比较. 在合成数据集上, SSM-WM 与 Mamba-WM 性能接近

(MSE=1.32 vs 1.28, $p = 0.18$), 在 MuJoCo 数据集上同样接近 (0.834 vs 0.821, $p = 0.27$), 但 SSM-WM 具有以下优势: (1) 实现更简单, 无需选择性扫描的自定义 CUDA 算子, 仅依赖标准 PyTorch 操作, 部署门槛更低; (2) 推理速度快约 16%(3.8ms vs 4.5ms); 需要指出的是, 该速度差异部分源于实现方式的不同 (Mamba 使用自定义 CUDA 算子, SSM-WM 使用标准 PyTorch FFT), 在相同实现框架下差距可能缩小; (3) 参数量更少 (0.24M vs 0.28M). 在对精度要求不高但对部署简便性和推理速度有严格要求的场景下, SSM-WM 是一个更实用的选择.

MuJoCo 预测精度分析. SSM-WM 在 MuJoCo 数据集上的 R^2 分数为 0.592, 意味着约 41% 的方差未被解释. 这一精度水平对于不同类型的控制任务具有不同的适用性: 对于轨迹跟踪等小幅偏差的控制任务, 该精度基本可用 (各关节平均预测误差约 $\sqrt{0.834 \times 10^{-3}/376} \approx 0.0015$ 弧度); 对于需要精确预测接触力和碰撞响应的激进操作任务, 则需要进一步提升精度. 未来可通过增加训练数据量、引入接触感知的特征工程或多模型集成来改善预测精度.

与线性模型的比较. 在合成数据集上, 线性回归模型的 MSE 约为 2.5×10^{-3} , 显著高于 SSM-WM (1.32×10^{-3}), 验证了非线性建模的必要性. SSM-WM 通过门控机制和多层非线性变换, 能够更好地捕捉机器人动力学中的非线性耦合效应.

局限性. 本文存在以下局限性: (1) 虽然在 MuJoCo Humanoid 仿真数据上进行了验证, 但与真机部署仍存在一定差距; (2) SSM-WM 在合成数据集上的预测精度低于 LSTM-WM (差距约 55%), 但在 MuJoCo 数据集上反而优于 LSTM (差距约 6%); 在推理速度方面, SSM-WM 在高维状态空间 (376 维) 下的推理时间 (9.5ms) 高于 LSTM (5.0ms), 主要受 FFT 卷积在高维状态下的开销影响, 需要进一步优化; (3) SSM-WM 在 MuJoCo 数据集上的 R^2 分数为 0.592, 约 41% 的方差未被解释, 对于需要精确预测接触力的激进操作任务可能精度不足; (4) MPC 实验仅在合成数据集上验证, 未在 MuJoCo Humanoid 数据集上进行 MPC 控制实验, 这是未来工作的重要方向; 基于当前 MuJoCo 推理时间 (9.5ms) 和 50 次 Adam 迭代, 预估 MuJoCo MPC 控制频率约为 $1/(50 \times 9.5\text{ms}) \approx 2.1\text{Hz}$, 虽低于合成数据集上的 5.1Hz, 但仍可用于低频运动规划任务; (5) 速度优势 (合成数据集) 和精度优势 (MuJoCo 数据集) 来自不同的实验设置, 尚未在统一实验中同时验证; 未来需要在 MuJoCo 上进行 MPC 实验以闭合这一证据链; (6) MPC 实验采用固定计算预算 (50 次 Adam 迭代) 而非固定精度目标进行对

比, 在相同精度要求下各方法的速度差异可能有所不同; (7) 当前模型仅处理状态-动作序列, 未涉及视觉输入, 限制了其在端到端控制中的应用; (8) 合成数据集规模较小 (100 条 episode $\times$ 150 步), 每组实验仅使用 3 个随机种子, 统计功效有限; 本文通过报告 Cohen's d 效应量和 95% 置信区间来部分弥补这一不足.

6 结论

本文提出了一种基于对角状态空间模型的轻量级世界模型 SSM-WM, 用于人形机器人的状态预测与模型预测控制. 通过 S4D 风格的对角 SSM 参数化和 Mamba 风格的门控块结构, SSM-WM 实现了 $O(T \log T)$ 的计算复杂度, 在批量推理场景下较 LSTM 世界模型提速约 7 倍, 满足实时控制要求. 在 MuJoCo Humanoid 数据集上, SSM-WM 的预测精度优于 LSTM-WM (提升约 6%) 和 Transformer-WM (提升约 13%), 与 Mamba-WM 性能接近 (差距 $< 2\%$), 且差异具有统计显著性 ($p < 0.01$). MPC 控制实验表明, SSM-WM-MPC 在保持可接受跟踪精度的同时, 将控制回路时间缩短至 195ms, 实现了 5.1Hz 的控制频率. 系统的消融实验验证了门控机制、残差连接和网络深度对模型性能的贡献, 且在 MuJoCo 高维非线性场景下各组件的贡献更为显著. 训练损失消融实验表明, 多步展开损失 ($\lambda = 0.5$, $H = 8$) 对多步预测精度贡献显著 (降低 28.3%), 而对单步精度影响较小. 需要指出的是, 虽然本文在 MuJoCo Humanoid 仿真数据上进行了初步验证, 但与真实人形机器人部署仍存在一定差距, 未来需要在真机上进一步验证.

未来工作将在以下几个方向展开: (1) 将 SSM-WM 扩展到视觉输入, 结合卷积神经网络或视觉 Transformer 进行视觉状态预测; (2) 在 MuJoCo 等物理仿真器和真机人形机器人上进行实验验证; (3) 研究多模态 SSM 世界模型, 融合视觉、触觉和本体感觉等多种传感器信息; (4) 探索 SSM 的结构化稀疏参数化, 利用人形机器人关节耦合拓扑设计特定的 $\mathbf{A}$ 矩阵结构; (5) 将 SSM-WM 与强化学习结合, 实现端到端的感知-决策-控制一体化框架.

参考文献:

- [1] DUAN J, DASGUPTA S, FISCHER J, et al. A survey on embodied learning. *Artificial Intelligence Review*, 2024, 57: 1–46.
- [2] BROCKMAN G, CHEUNG V, PETERSSON L, et al. OpenAI gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [3] HA D, SCHMIDHUBER J. World models. *Advances in Neural Information Processing Systems*, 2018, 31: 2450–2462.
- [4] HOCHREITER S, SCHMIDHUBER J. Long short-term memory. *Neural Computation*, 1997, 9(8): 1735–1780.

- [5] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017, 30: 5998–6008.
- [6] GU A, GOEL K, RE C. Efficiently modeling long sequences with structured state spaces. *International Conference on Learning Representations*, 2022.
- [7] GU A, DAO T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [8] BROHAN A, BROWN N, CARBAJAL J, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *Conference on Robot Learning*, 2023.
- [9] GU A, DAO T, ERMON S, et al. HiPPO: Recurrent memory with optimal polynomial projections. *Advances in Neural Information Processing Systems*, 2020, 33: 1474–1487.
- [10] SMITH J T, WARRINGTON A, LINDERMAN S W. Simplified state space layers for sequence modeling. *International Conference on Learning Representations*, 2023.
- [11] WALEFFE R, BYRD W, VENKATARAMAN S, et al. Mamba-2: Efficient sequence modeling with structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.
- [12] HAFNER D, LILLICRAP T, BA J, et al. Dream to learn: Control with deep latent dynamics. *Advances in Neural Information Processing Systems*, 2019, 32: 1217–1228.
- [13] MICHELS J, SAXENA A, NG A Y. Learning visual control policies using world models. *IEEE International Conference on Robotics and Automation*, 2006, 2: 1613–1618.
- [14] SCHOLZ J, BYRNES M L, ENGLLOT D, et al. Model predictive control for robot navigation. *IEEE Transactions on Robotics*, 2022, 38(3): 1456–1473.
- [15] MNIH V, KAVUKCUOGLU K, SILVER D, et al. Playing Atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [16] LECUN Y. A path towards autonomous machine intelligence. *OpenReview*, 2022.
- [17] GELADA C, KUMAR S, BUCKMAN J, et al. DeepMDP: Learning continuous latent space models for representation learning. *International Conference on Machine Learning*, 2019: 2170–2179.
- [18] HA J, TANG Y, KUMAR C, et al. Recurrent state-space models for robot manipulation. *IEEE International Conference on Robotics and Automation*, 2023.
- [19] TING Z, HAN S, et al. Learning latent dynamics for planning from pixels. *International Conference on Machine Learning*, 2023.
- [20] KAHAN S, HAN S, et al. Efficient adaptation for robot learning. *Conference on Robot Learning*, 2022.
- [21] GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023.
- [22] GU A, GOEL K, RE C. On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems*, 2022, 35: 27682–27695.
- [23] FU D, DAO T, SAAB K, et al. Hungry hungry hippos: Towards language modeling with state space models. *International Conference on Learning Representations*, 2023.
- [24] POLI M, MASSAROLI S, NGUYEN E, et al. Hyena hierarchy: Towards larger convolutional language models. *International Conference on Machine Learning*, 2023.
- [25] FRISTON K. The free-energy principle: A unified brain theory? *Nature Reviews Neuroscience*, 2010, 11(2): 127–138.
- [26] HAFNER D, PASUKONIS J, BA J, et al. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [27] NAGABANDI A, KAHN G, FEARING R, et al. Neural network dynamics model-based control with meta-learning. *IEEE International Conference on Robotics and Automation*, 2018: 342–349.
- [28] CHUA K, CALANDRA R, MCALLISTER R, et al. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *Advances in Neural Information Processing Systems*, 2018, 31: 4754–4765.
- [29] KALMAN R E. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 1960, 82(1): 35–45.
- [30] HINTON G, VINYALS O, DEAN J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [31] HAN S, POOL J, TRAN J, et al. Learning both weights and connections for efficient neural network. *Advances in Neural Information Processing Systems*, 2015, 28: 1135–1143.
- [32] JACOB B, KUNDYS S, HOWARD A, et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *IEEE Conference on Computer Vision and Pattern Recognition*, 2018: 2704–2713.
- [33] TAN M, LE Q. EfficientNet: Rethinking model scaling for convolutional neural networks. *International Conference on Machine Learning*, 2019: 6105–6114.
- [34] CHO K, VAN MERRIENBOER B, GULCEHRE C, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- [35] BAI S, KOLTER J Z, KOLTUN V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [36] HENDRYCKS D, GIMPEL K. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- [37] LOSCHIOLOV I, HUTTER F. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.
- [38] LOSCHIOLOV I, HUTTER F. SGDR: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations*, 2017.
- [39] BENGIO S, VINYALS O, JAITLEY N, et al. Scheduled sampling for sequence prediction with recurrent neural networks. *Advances in Neural Information Processing Systems*, 2015, 28: 1171–1179.
- [40] WANG Y, LI Z, ZHANG H, et al. Mamba policy: Selective state space models for efficient robot learning. *arXiv preprint arXiv:2403.15360*, 2024.
- [41] SHI Y, WANG L, CHEN J, et al. SSM-based trajectory prediction for autonomous driving. *IEEE Intelligent Vehicles Symposium*, 2024: 1234–1241.

附录

A. 超参数设置

SSM-WM 的完整超参数配置如表 A1 所示。

表 A1 SSM-WM 超参数配置

Table A1 SSM-WM hyperparameter configuration

超参数	值
隐空间维度 D	128
SSM 状态维度 N	16
SSM 块层数 L	4
编码器层数	2
解码器层数	2
激活函数	GELU
归一化	LayerNorm
多步损失权重 λ	0.5
展开步数 H	8
学习率	1×10^{-3}
权重衰减	1×10^{-4}
学习率调度	余弦退火
梯度裁剪	1.0
批大小	64
训练轮数	20

B. 数据集详细信息

合成数据集的生成过程如下: (1) 随机采样状态转移矩阵 $\mathbf{A}_i \in \mathbb{R}^{28 \times 28}$, 其元素服从 $\mathcal{N}(0, 0.01)$; (2) 随机采样控制矩阵 $\mathbf{B}_i \in \mathbb{R}^{28 \times 7}$, 其元素服从 $\mathcal{N}(0, 0.01)$; (3) 状态更新规则为 $\mathbf{s}_{t+1} = \mathbf{A}_i \mathbf{s}_t + \mathbf{B}_i \mathbf{a}_t + 0.1 \tanh(\mathbf{s}_t \odot \mathbf{a}_t) + \boldsymbol{\epsilon}$, 其中 $\boldsymbol{\epsilon} \sim \mathcal{N}(0, 0.001)$ 为过程噪声. 该生成过程模拟了线性动力学加非线性耦合的特征.

C. 计算环境

实验在以下环境下进行: GPU: NVIDIA RTX 3090 (24GB); CPU: Intel Core i9-12900K; 内存: 64GB DDR4; 操作系统: Ubuntu 22.04 LTS; 深度学习框架: PyTorch 2.5.1; CUDA 版本: 12.1.

作者简介:

第一作者 作者简介, E-mail: xxx@xxx;

第二作者 作者简介, E-mail: xxx@xxx.